

Going deeper: the mechanics of PyTorch

Computation graphs · autograd · nn.Module · custom layers · feature columns

Running example: a warehouse AGV fleet

Q1 will this approach dock? — a 2-feature problem no straight line can solve

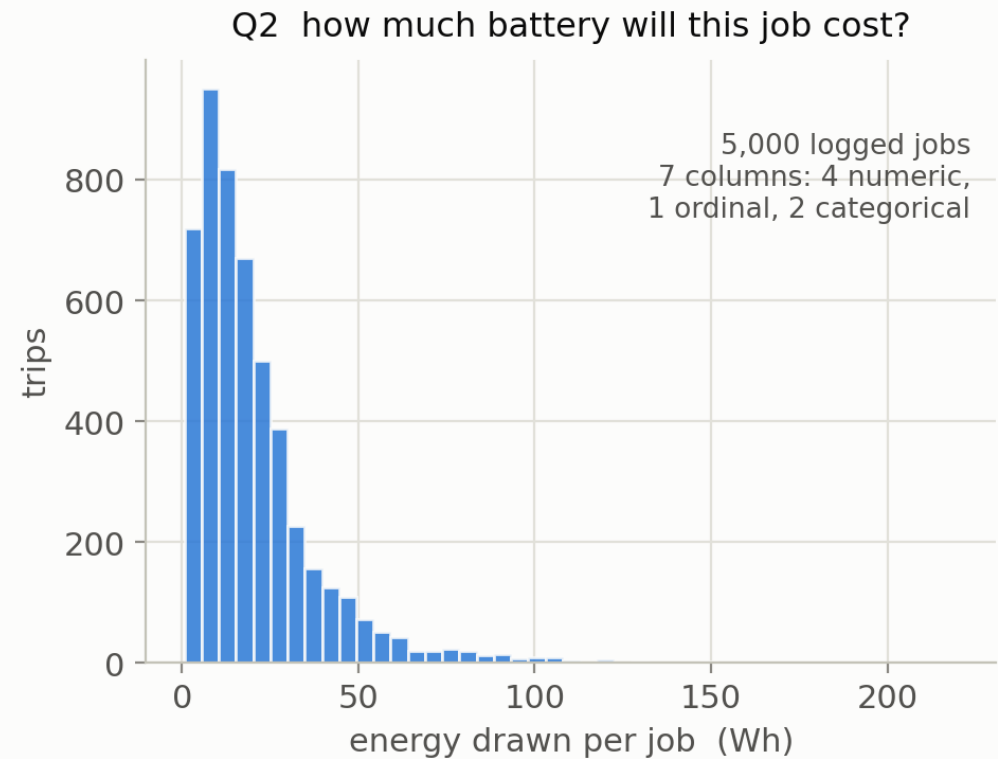
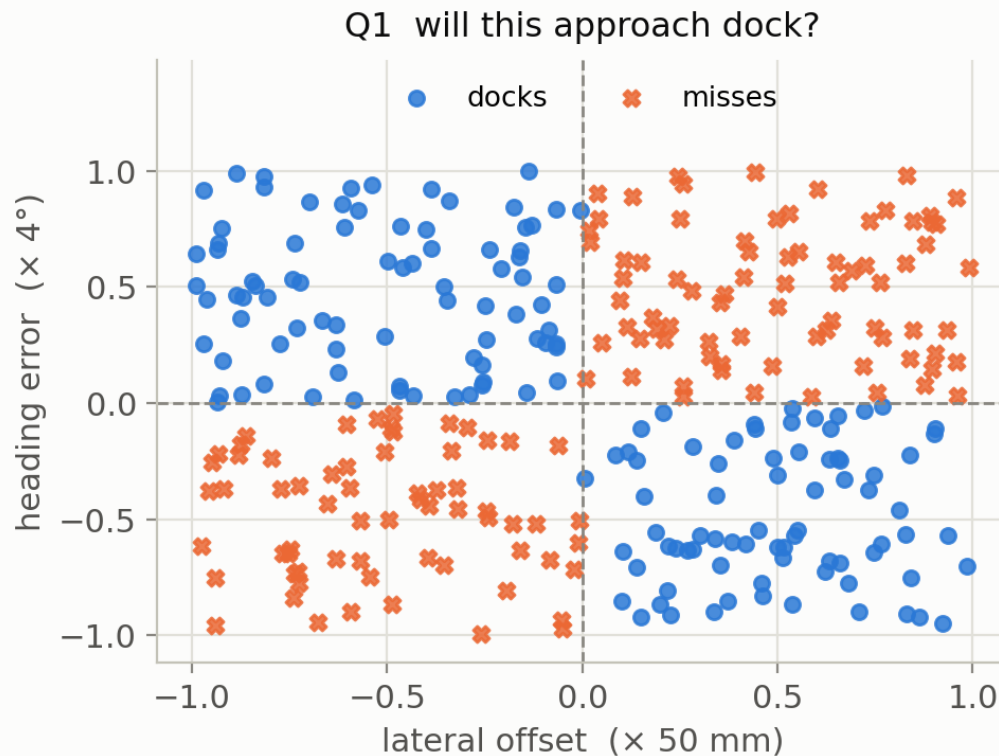
Q2 how many watt-hours will this job draw? — 5 000 jobs, 7 mixed-type columns

Companion notebook: [mlpi26_Lect08_nbsup1.ipynb](#)

Upper-undergraduate engineering · every number on these slides is computed in that notebook

One AGV fleet, two questions

- Q1 asks whether a vehicle whose pose is slightly wrong will still dock — two inputs, a yes/no answer, and a decision boundary that will turn out to be XOR-shaped.
- Q2 asks what a transport job will cost the battery — 5 000 logged jobs with continuous, ordinal and categorical columns that all have to become one tensor.

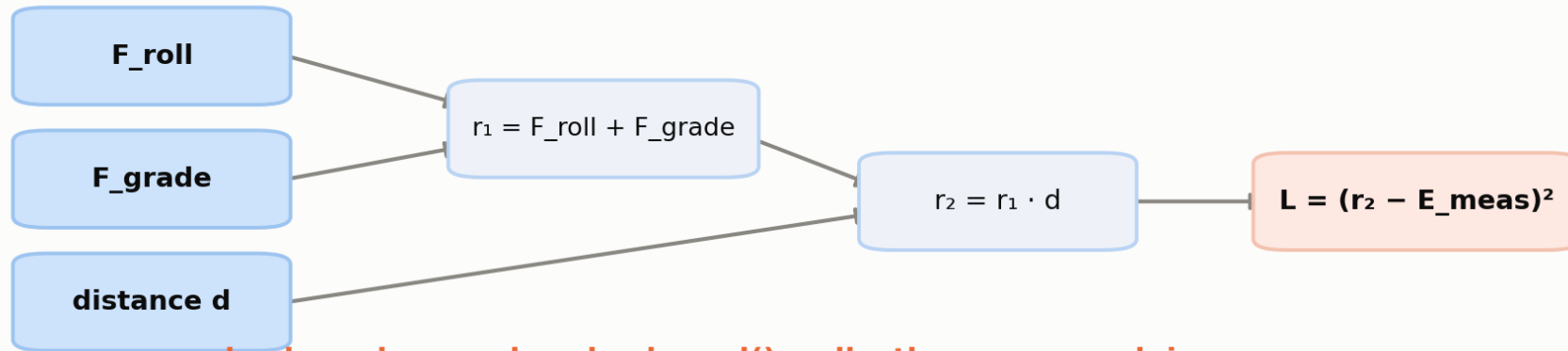


Between them these two exercise every mechanism in the chapter: the graph, autograd, Sequential vs Module, capacity, custom layers and feature columns.

PyTorch records a graph as your code runs

- Every tensor operation runs immediately and records a node, so the graph is a by-product of ordinary Python — there is no separate declaration or compile step.
- Dynamic means you may put if, for and print inside a model and step through it in a debugger: the reason PyTorch reads like NumPy.

forward — the graph is recorded line by line, as Python runs



backward — one `loss.backward()` walks the same graph in reverse

leaf tensors with `requires_grad=True` come back holding `.grad`; the graph is then freed and rebuilt next iteration

```
def traction_work(f_roll, f_grade, distance):  
    r1 = torch.add(f_roll, f_grade)  
    return torch.mul(r1, distance)  
  
# plain Python, no decorators  
# works for rank 0, 1 or 2 inputs  
# ... and for a whole batch
```

Four attributes decide what a tensor can do

- Get any of the four wrong and you get a specific, recognisable error — except `requires_grad`, whose failure is silent: no `.grad`, so the optimiser updates nothing.

shape	dtype	device	requires_grad
(64, 14)	torch.float32	cpu / cuda:0	True
64 trips × 14 features	gradients need floats	both operands must match	only then is a graph recorded

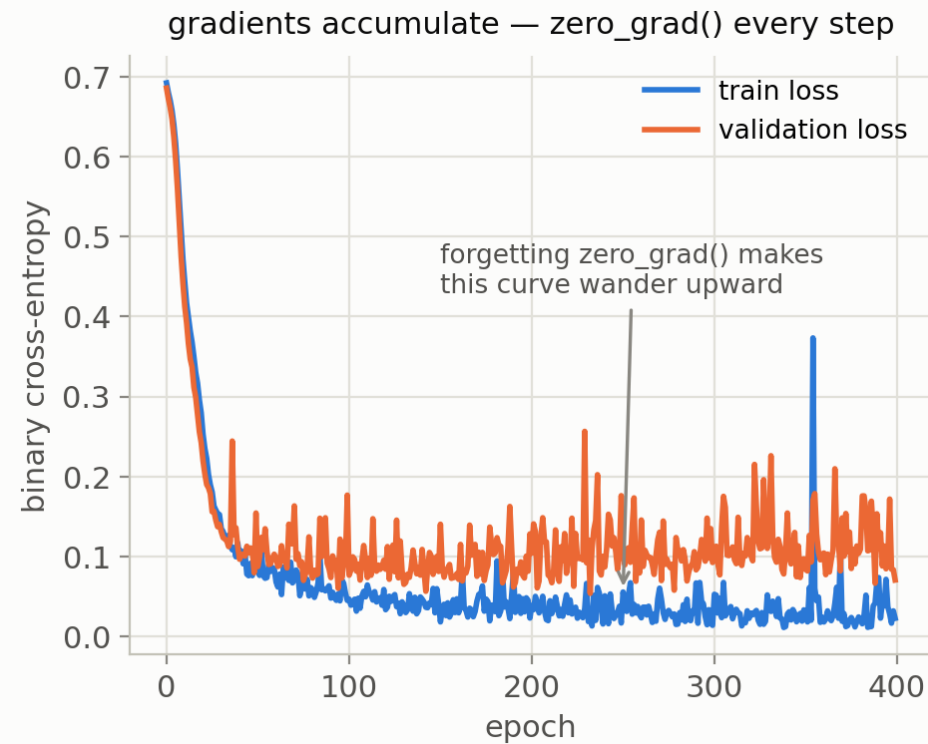
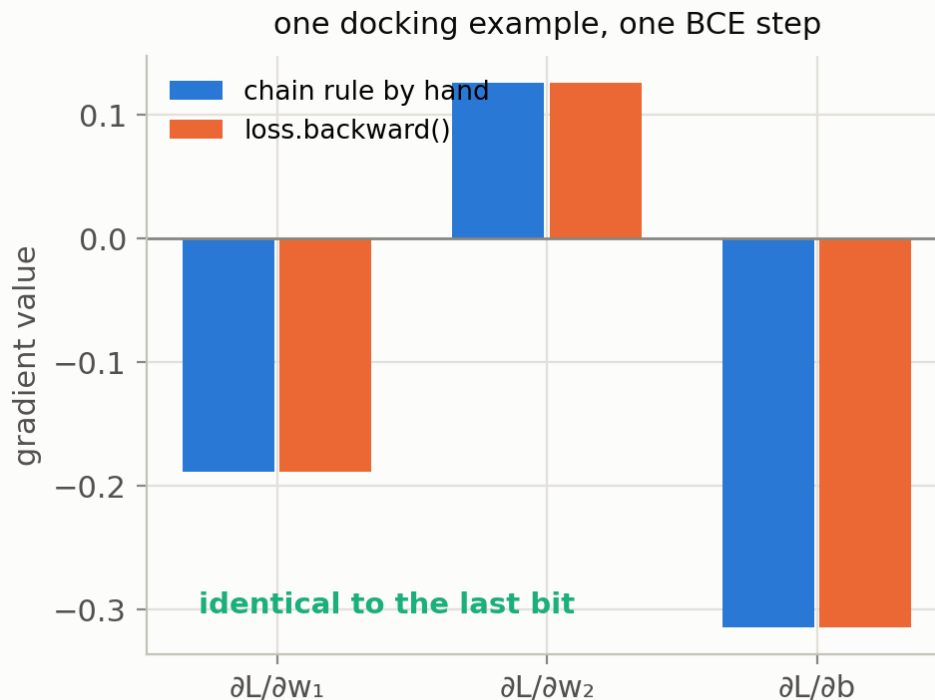
a leaf tensor with `requires_grad=True` is a model parameter;
everything computed from it carries a `grad_fn` and is not a leaf

```
self.w1 = nn.Parameter(nn.init.xavier_normal_(torch.empty(2, 8))) # registered
self.w2 = torch.empty(2, 8, requires_grad=True)                  # invisible!
# -> a bare tensor never reaches model.parameters(), so it never trains
```

Xavier/Glorot initialisation draws weights with variance $2/(n_{in} + n_{out})$ so activations and gradients keep the same scale layer after layer.

One backward() call, every gradient

- For a sigmoid output with cross-entropy loss the algebra collapses to $\partial L / \partial z = p - y$, so $\partial L / \partial w = (p - y)x$ — and autograd reproduces it exactly.



Gradients accumulate: .grad is added to, never overwritten. optimizer.zero_grad() once per step, and torch.no_grad() around everything that is not training.

nn.Sequential, and choosing the loss

- Sequential chains modules in the order you list them, and stays indexable so you can initialise or regularise one layer at a time.

```
model = nn.Sequential(nn.Linear(2, 8), nn.ReLU(),  
                      nn.Linear(8, 8), nn.ReLU(),  
                      nn.Linear(8, 1), nn.Sigmoid())  
nn.init.xavier_uniform_(model[0].weight)  
l1_penalty = 1e-3 * model[2].weight.abs().sum()
```

105 parameters
one layer by index
add it to the loss

the loss follows the task

regression → no output activation + nn.MSELoss

binary → nn.BCEWithLogitsLoss on raw logits

multi-class → nn.CrossEntropyLoss on raw logits

CrossEntropyLoss applies log-softmax itself — adding a Softmax layer applies it twice and flattens the gradients.

the optimizer follows your patience

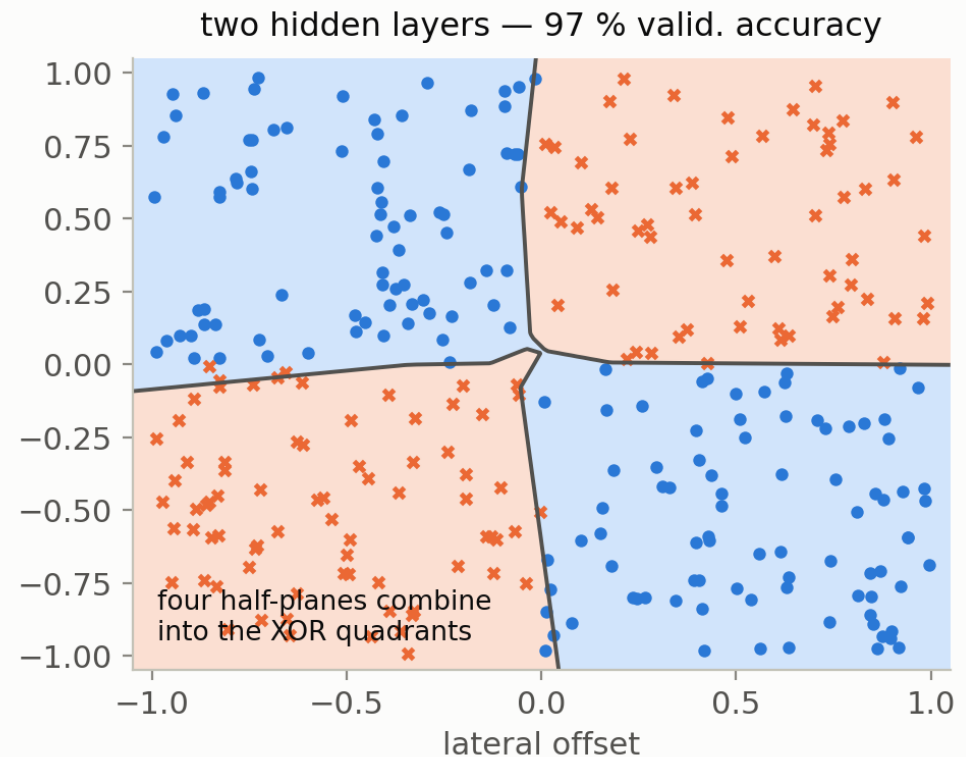
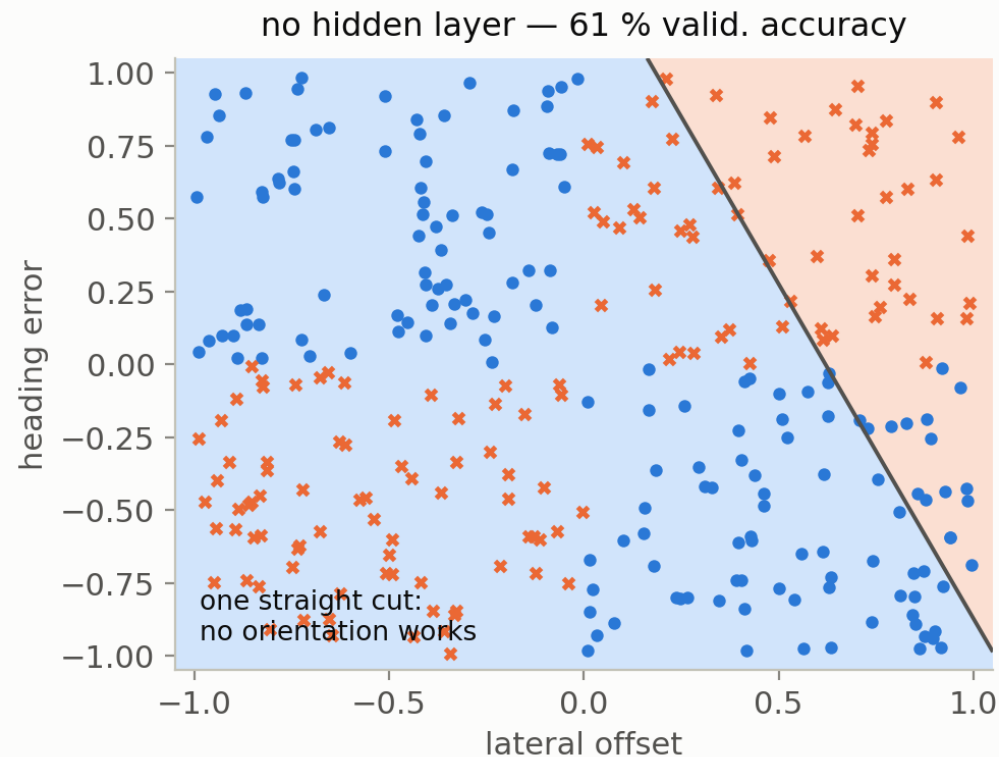
SGD — one learning rate to tune, transparent behaviour, used here for the docking model

Adam — per-parameter step sizes, forgiving defaults, used here for the energy regression

L2 regularisation is the weight_decay argument; L1 you add to the loss by hand.

A straight line cannot dock an AGV

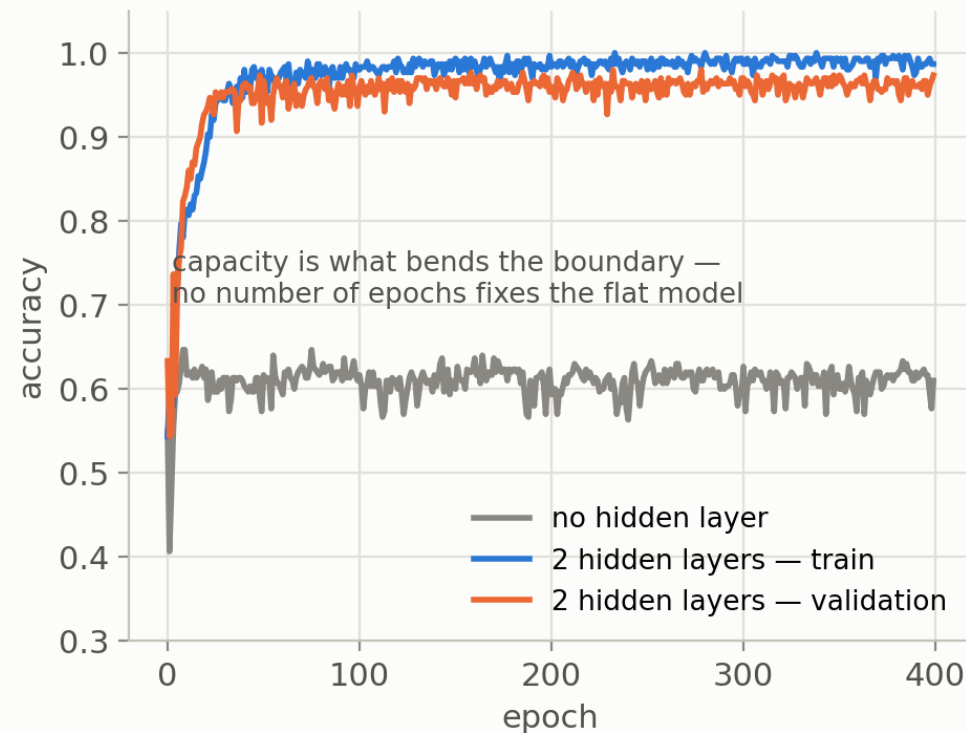
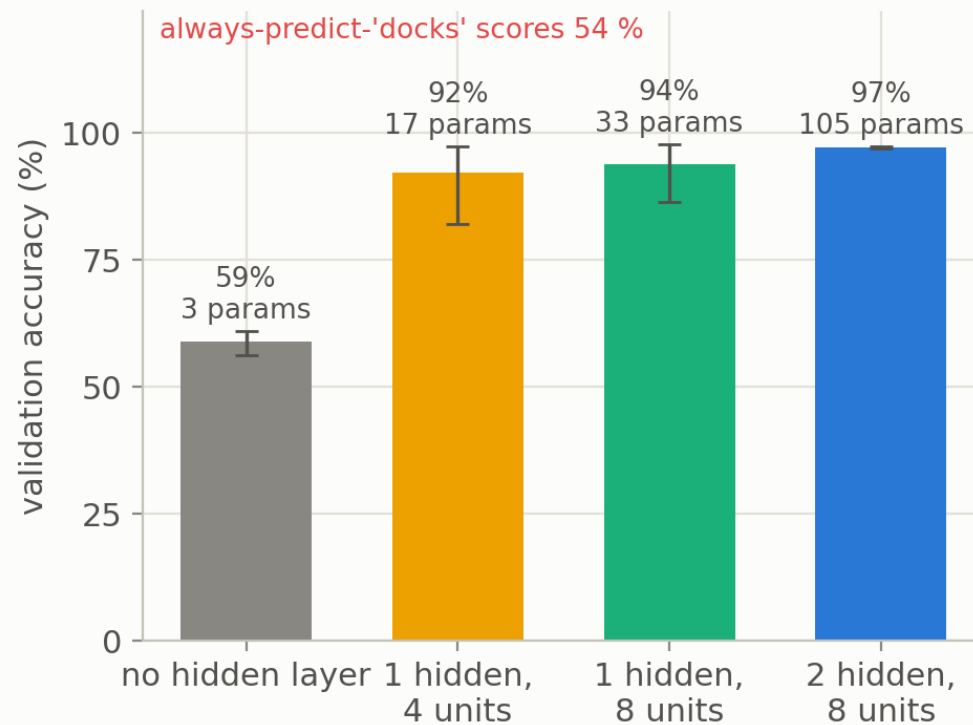
- Logistic regression on the two pose errors reaches 61 % validation accuracy — against 54 % for always predicting "docks" — and its loss sits at $\ln 2 = 0.693$ for all 400 epochs.



Two hidden layers and 105 parameters reach 97 %: four half-planes intersect into the XOR quadrants. Capacity, not training time, is what bends a boundary.

How much model is actually enough?

- One hidden layer already bends the boundary — the universal approximation theorem in practice. Depth buys the same accuracy with fewer parameters and far less seed-to-seed variance.



Read the diagnosis off the curves: high and flat train loss means too little capacity; a rising validation curve means too much. They call for opposite fixes.

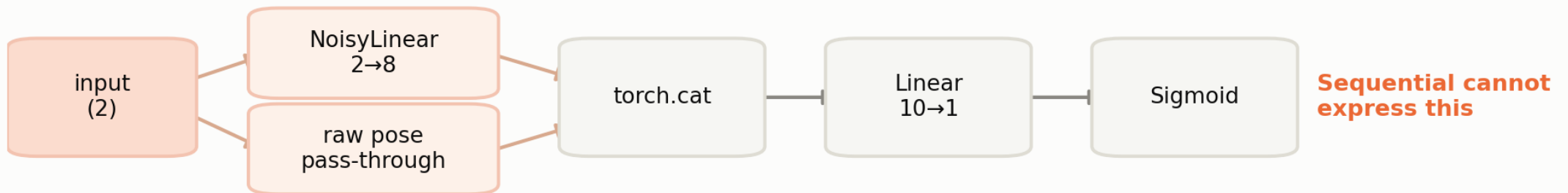
nn.Module: you write forward(), so it can branch

- `__init__` declares the sub-modules — that is what registers their parameters — and `forward()` says how the data flows. Call the module, never `forward()` directly.

nn.Sequential — one straight chain, written as a list



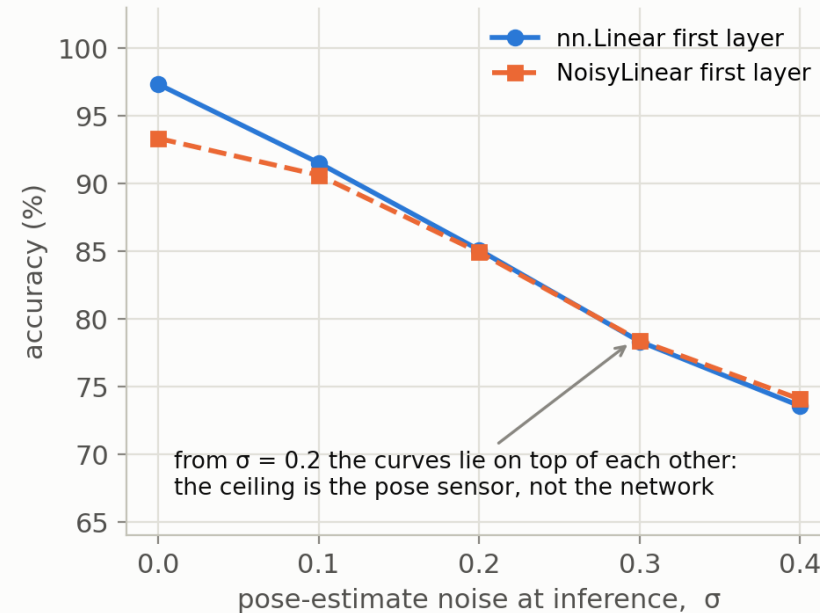
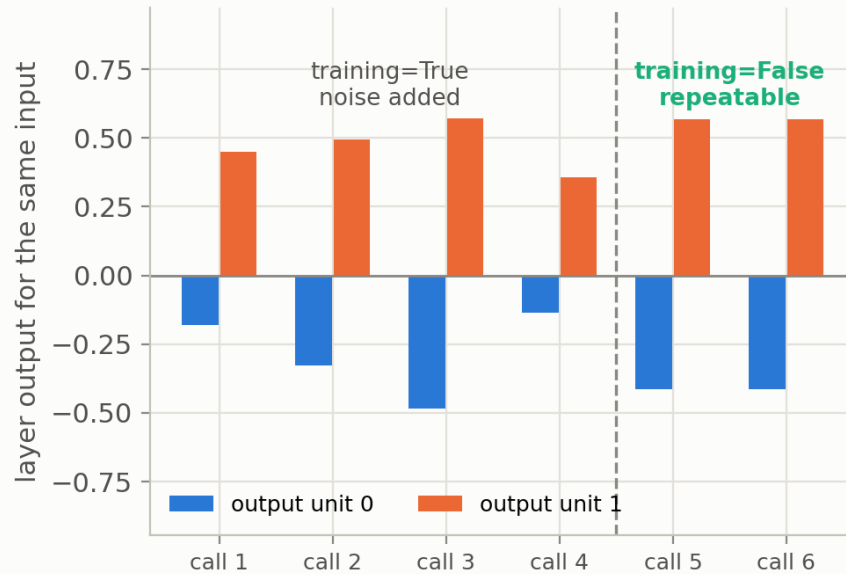
nn.Module — you write forward(), so the data can branch and rejoin



Subclass as soon as the data branches, rejoins, is reused, or flows differently in training than in inference. Build layers in a loop with `nn.ModuleList` — a plain list registers nothing.

Writing a layer that PyTorch does not have

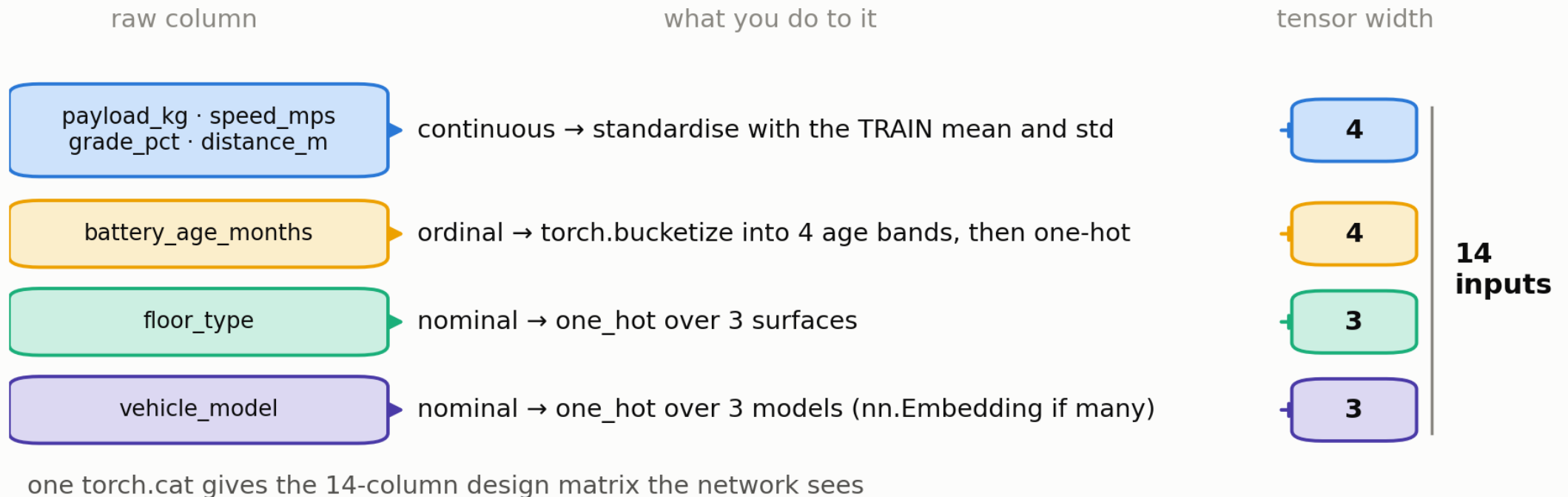
```
class NoisyLinear(nn.Module):
    #  $w.T (x + \epsilon) + b$ 
    def forward(self, x):
        if self.training:
            # set by .train() / .eval()
            x = x + torch.normal(0., self.noise_stddev, x.shape)
        return torch.add(torch.mm(x, self.w), self.b)
```



Honest result: injecting pose noise did not help. From $\sigma = 0.2$ both networks sit on the same curve — the 78 % ceiling belongs to the sensor, not the architecture.

Getting mixed columns into one tensor

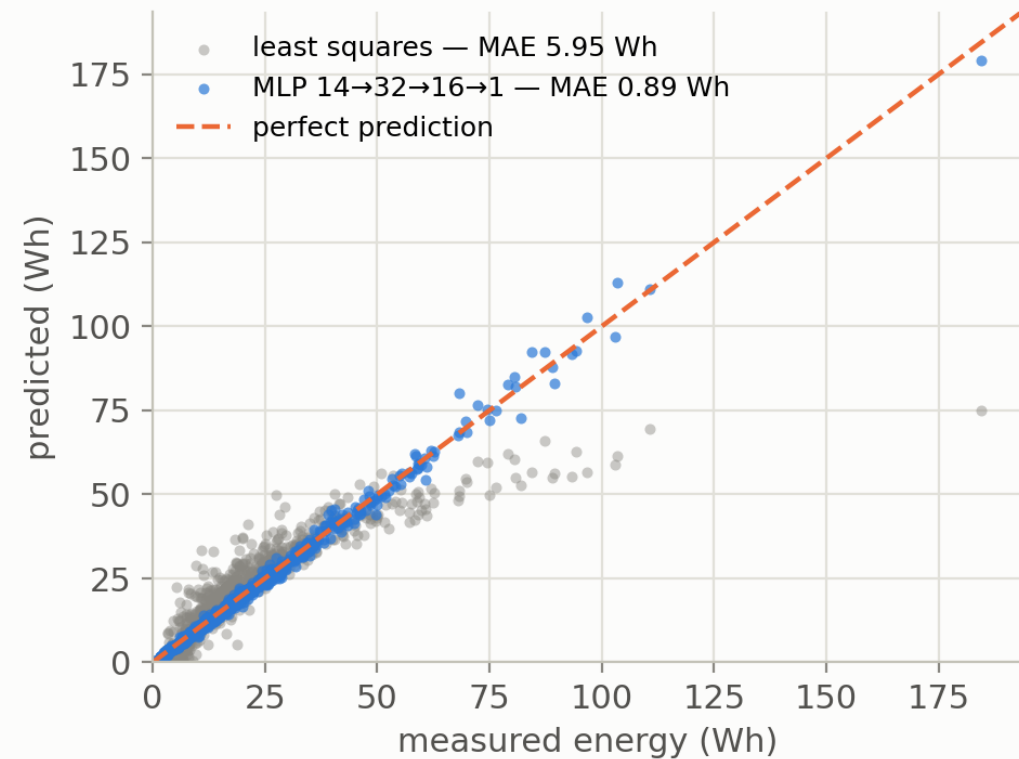
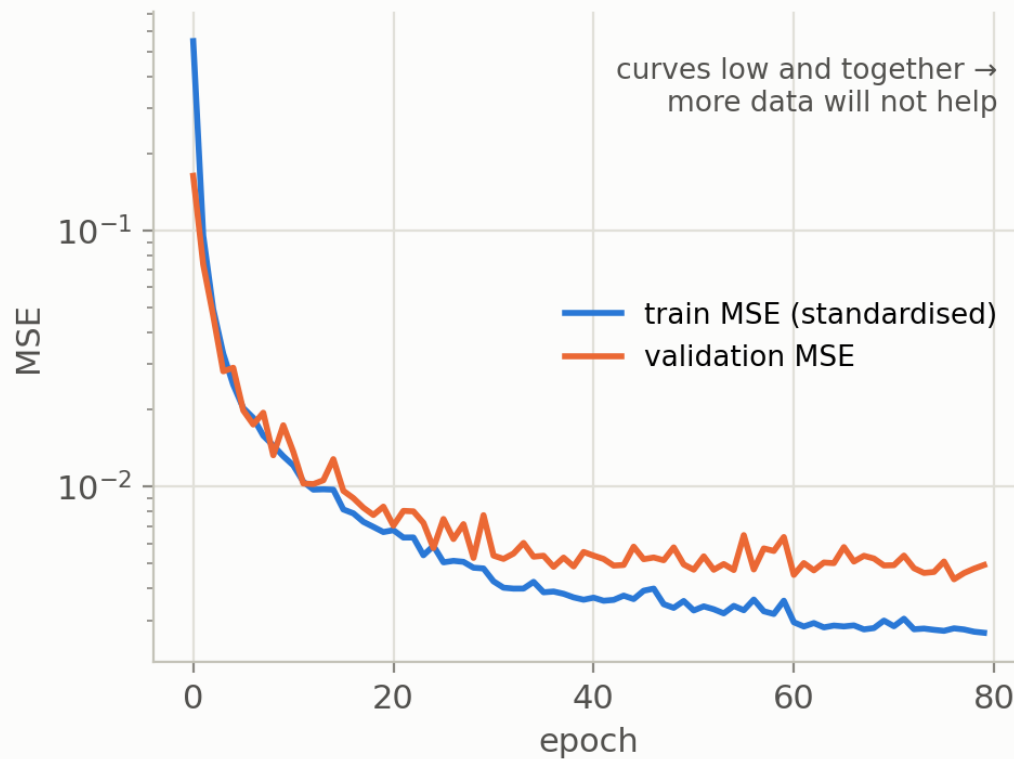
- Continuous, ordinal and nominal columns each need a different conversion; one `torch.cat` then produces the design matrix the network sees.



Split before you compute any statistic. The mean and standard deviation come from the training rows only — using the whole dataset leaks the test set into training and inflates your score.

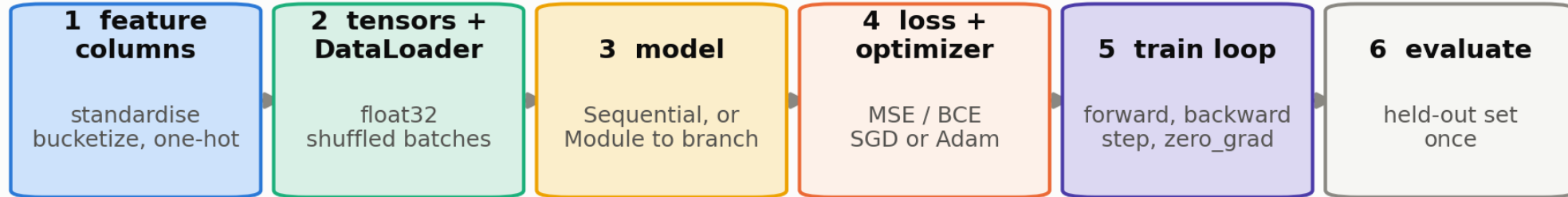
End to end: what will this job cost?

- A 1 025-parameter MLP on the 14 encoded columns reaches 0.89 Wh mean absolute error on 750 held-out jobs, against 5.95 Wh for least squares on exactly the same features.



R^2 rises from 0.760 to 0.9941, and 0.89 Wh is within a factor of 1.3 of the 0.71 Wh measurement noise floor — there is almost nothing left to win.

What to take away



every PyTorch project you will write follows this shape

1. The graph is recorded as your Python runs; `backward()` walks it in reverse. Nothing is declared in advance.
2. Parameters must be `nn.Parameter` or sub-modules — a bare tensor or a plain list never reaches the optimizer.
3. Gradients accumulate. `zero_grad()` every step; `no_grad()` around everything that is not training.
4. `Sequential` for a chain, subclass `nn.Module` the moment the data branches or the layer has a training mode.
5. High-and-flat train loss means capacity; a rising validation curve means regularisation. Opposite fixes.
6. Split first, then fit every preprocessing statistic on the training rows only.
7. Always beat a dumb baseline on the same features, quote the noise floor, and touch the test set once.

Generated by Claude
and customized by



dew.ninja 2026